THE UNIVERSITY OF AZAD JAMMU AND KASHMIR

Department of Software Engineering | Semester 5th | Session 2024-2028

STUDENT NAME

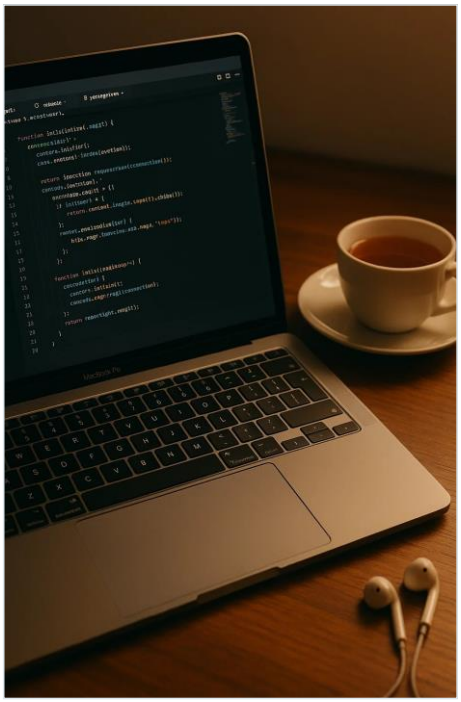
Tayyab Hanif Awan

COMPLETE ROLL NUMBER

2024-SE-11

DIP LAB REPORT 02

Digital Images, Pixels, Color and Basic Transformations



SOURCE IMAGE

Leisure.jpg

Format: Color JPEG

Resolution: 1024 x 1536

Channels: 3 (BGR in OpenCV)

Data type: uint8

Input: Student-selected image

SUBMISSION DETAILS

Course: Digital Image Processing Lab

Instructor: Engr. Fahad Nisar

Lab objective: Load and inspect a color image, analyze RGB channels and pixel intensities, crop and resize regions, produce a grayscale histogram, and compare grayscale bit depths.

Files delivered separately: `lab02.py`, source image, generated PNG outputs, `.gitignore`, and this report.

01 / ENVIRONMENT AND SETUP

Virtual environment and libraries

Evidence from the attached screenshots - Python, OpenCV, NumPy and Matplotlib

The project was created in dip-lab-02 with an isolated .venv. Required packages were installed inside the environment, followed by an import and Python-executable check.

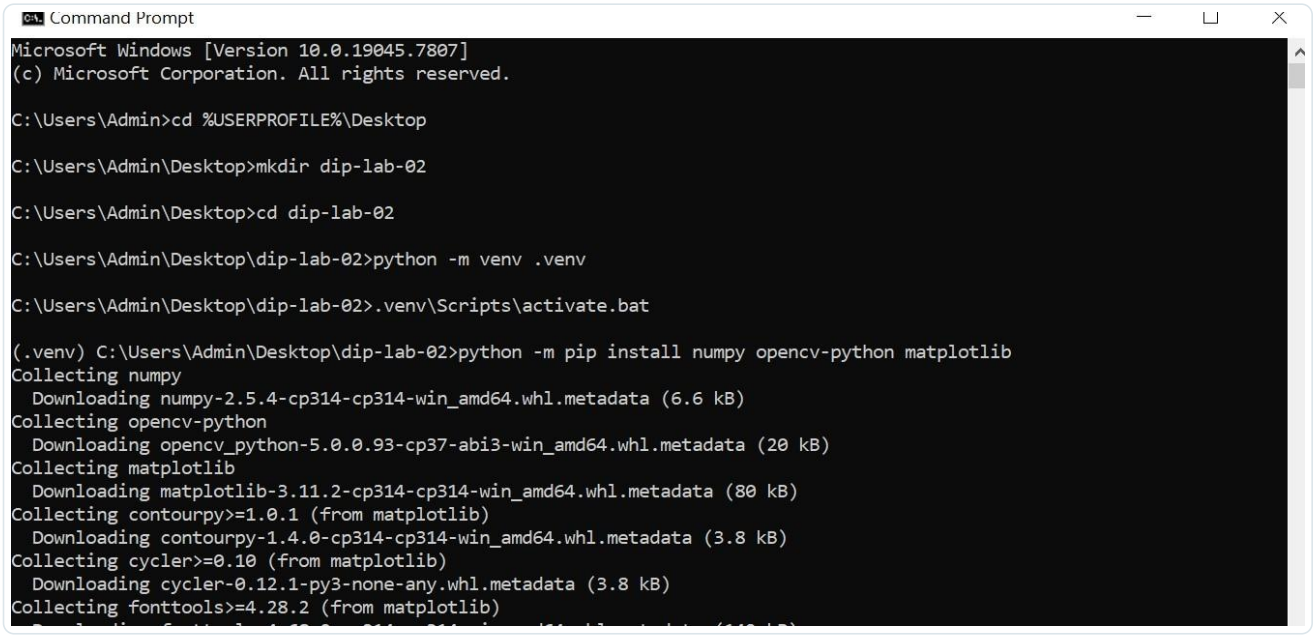


Figure. Student terminal screenshot: project creation, virtual-environment activation and package installation.

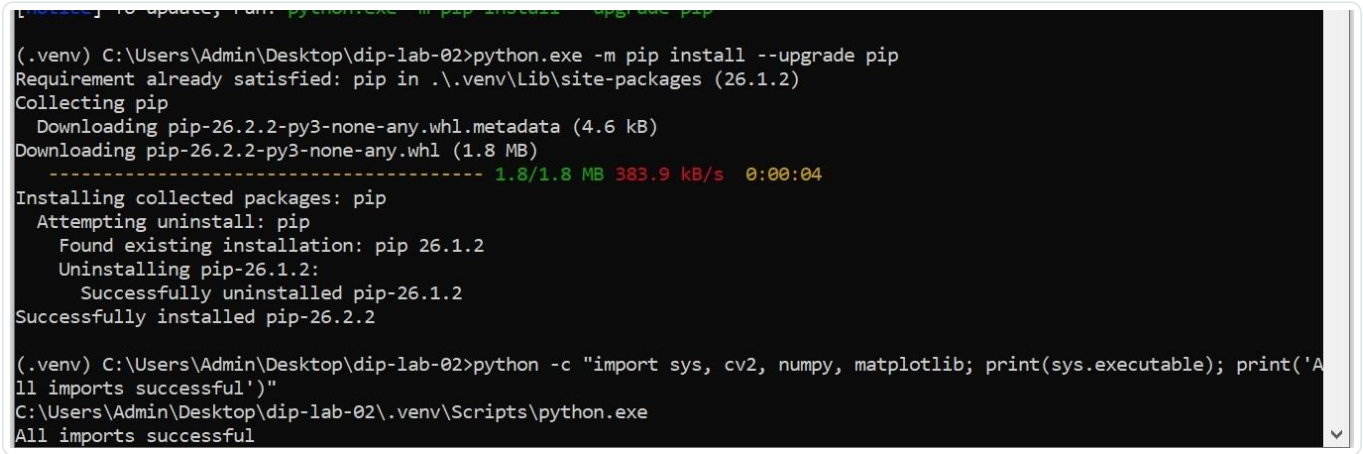


Figure. Student terminal screenshot: packages installed and imports verified using the .venv Python executable.

COMMANDS USED - STEP 1

```
python -m venv .venv
.venv\Scripts\activate.bat
python -m pip install numpy opencv-python matplotlib
python -c "import sys,cv2,numpy,matplotlib; print(sys.executable)"
```

Result: The terminal shows `...\dip-lab-02\.venv\Scripts\python.exe` and the message All imports successful. The virtual environment is excluded from version control with `.venv/` in `.gitignore`.

02 / LOAD AND INSPECT

Image properties and pixel values

Steps 2-3 - validated image loading and [row, column] indexing

Input image	Leisure.jpg (JPG; color)
Image shape	(1536, 1024, 3)
Height x width	1536 x 1024 pixels
Data type	uint8; intensity range 0-255

```
=== Step 2: Image Loading ===
Input image: leisure.jpg
Image shape (height, width, channels): (1536, 1024, 3)
Image data type: uint8

=== Step 3: Image Inspection ===
Height: 1536
Width: 1024
Number of channels: 3
Top-left pixel [row=0, column=0] (BGR): [16, 29, 38]
Center pixel [row=768, column=512] (BGR): [36, 68, 102]
Global minimum intensity: 0
Global maximum intensity: 255
Blue channel minimum: 0; maximum: 255
```

Figure. Student terminal screenshot: loaded image, shape, dtype, top-left/center pixels and global/channel min-max values.

RELEVANT SOURCE CODE - STEPS 2 AND 3

```
image = cv.imread(str(IMAGE_PATH), cv.IMREAD_COLOR)
if image is None:
    raise FileNotFoundError(f"Could not load: {IMAGE_PATH}")
height, width, channels = image.shape
print("Image shape:", image.shape, "dtype:", image.dtype)
print("Top-left (BGR):", image[0, 0])
center_row, center_col = height // 2, width // 2
print("Center (BGR):", image[center_row, center_col])
print("Global min/max:", image.min(), image.max())
for k, name in enumerate(("Blue", "Green", "Red")):
    ch = image[:, :, k]
    print(name, "min/max:", ch.min(), ch.max())
```

Recorded values: The captured local run shows top-left BGR = [16, 29, 38], center BGR = [36, 68, 102], and global / per-channel extrema of 0 and 255. Indexing uses `[row, column]`, not (x, y).

Figures were reproduced using the separately uploaded JPEG; slight pixel-level differences from the local screenshot are possible when JPEG copies differ.

03 / COLOR AND CHANNELS

BGR, RGB and channel separation

Step 4 - six labeled views, grayscale and center-pixel HEX



Figure. Six-panel figure regenerated from the uploaded Leisure.jpg: RGB, red, green, blue, grayscale and unconverted BGR display.

Local center pixel (RGB)	[102, 68, 36]
Local center pixel (HEX)	#664424

RELEVANT SOURCE CODE - STEP 4

```
rgb = cv.cvtColor(image, cv.COLOR_BGR2RGB)
gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
red, green, blue = cv.split(rgb)
center_rgb = rgb[height // 2, width // 2]
center_hex = "#{:02X}{:02X}{:02X}".format(*center_rgb)
fig, axes = plt.subplots(2, 3, figsize=(13, 8))
views = [(rgb, "RGB", None), (red, "Red", "gray"),
         (green, "Green", "gray"), (blue, "Blue", "gray"),
         (gray, "Grayscale", "gray"), (image, "BGR", None)]
for ax, (data, title, cmap) in zip(axes.flat, views):
    ax.imshow(data, cmap=cmap)
    ax.set_title(title)
    ax.axis("off")
```

BGR versus RGB: OpenCV reads color images as blue, green, red (BGR), whereas Matplotlib displays three-channel arrays as red, green, blue (RGB). Converting first prevents visibly incorrect colors. Each isolated channel visualizes the intensity of one color component.

04 / SPATIAL TRANSFORMATIONS

Crop, resize and save

Step 5 - exact 100 x 100 crop enlarged to 200 x 200 pixels



Crop rows / columns	718:818 / 462:562
Crop shape	(100, 100, 3)
Resized shape	(200, 200, 3)
Save confirmation	cv.imwrite(...) returned True

RELEVANT SOURCE CODE - STEP 5

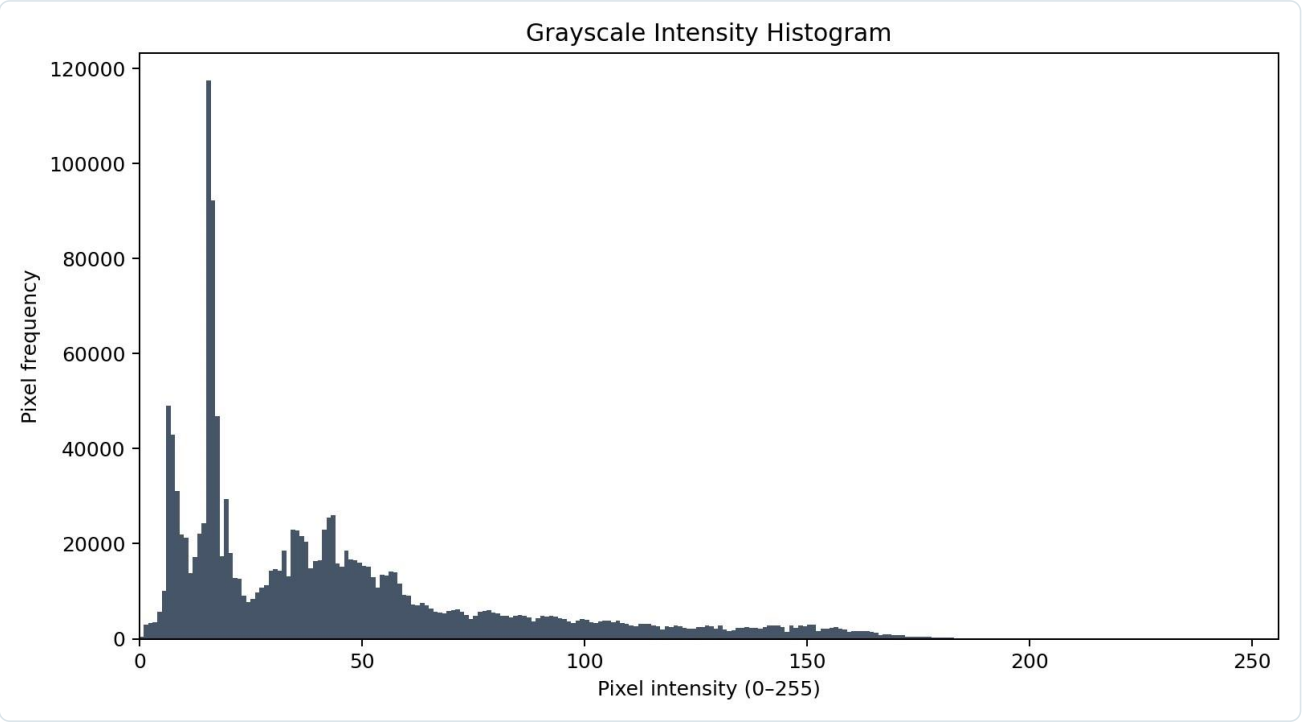
```
y_start = (height - 100) // 2
x_start = (width - 100) // 2
crop_rgb = rgb[y_start:y_start + 100,
               x_start:x_start + 100]
resized_rgb = cv.resize(crop_rgb, (200, 200),
                        interpolation=cv.INTER_LINEAR)
resized_bgr = cv.cvtColor(resized_rgb,
                          cv.COLOR_RGB2BGR)
saved = cv.imwrite("resized_crop.png", resized_bgr)
print("Crop shape:", crop_rgb.shape)
print("Resized shape:", resized_rgb.shape)
print("Saved:", saved)
```

Observation: The crop selects a smaller spatial region without changing its content. Resizing increases its pixel dimensions using interpolation, so it does not recover detail that was absent in the original crop.

05 / HISTOGRAM AND BRIGHTNESS

Grayscale intensity distribution

Step 6 - histogram with 256 bins over the interval [0, 256)



Mean intensity (local run)	45.25 / 255
Dark / midtone / bright	84.7% / 14.9% / 0.4%

Interpretation: Leisure.jpg is predominantly dark: about 84.7% of its pixels fall in the 0-84 range, while only 0.4% lie in the 171-255 range. The mean grayscale intensity of about 45.25/255 confirms the dominance of dark tones. The histogram measures how often each brightness value occurs, but it cannot show where those pixels are located or identify objects in the scene.

RELEVANT SOURCE CODE - STEP 6

```
flat_gray = gray.ravel()
fig_hist, ax_hist = plt.subplots()
ax_hist.hist(flat_gray, bins=256, range=(0, 256))
ax_hist.set_title("Grayscale Intensity Histogram")
ax_hist.set_xlabel("Pixel intensity (0-255)")
ax_hist.set_ylabel("Pixel frequency")
fig_hist.savefig("grayscale_histogram.png")
mean_intensity = np.mean(gray)
dark_pct = np.mean(gray < 85) * 100
bright_pct = np.mean(gray >= 171) * 100
```

06 / BIT DEPTH

Grayscale quantization

8-bit gradient versus 1-, 2- and 4-bit intensities

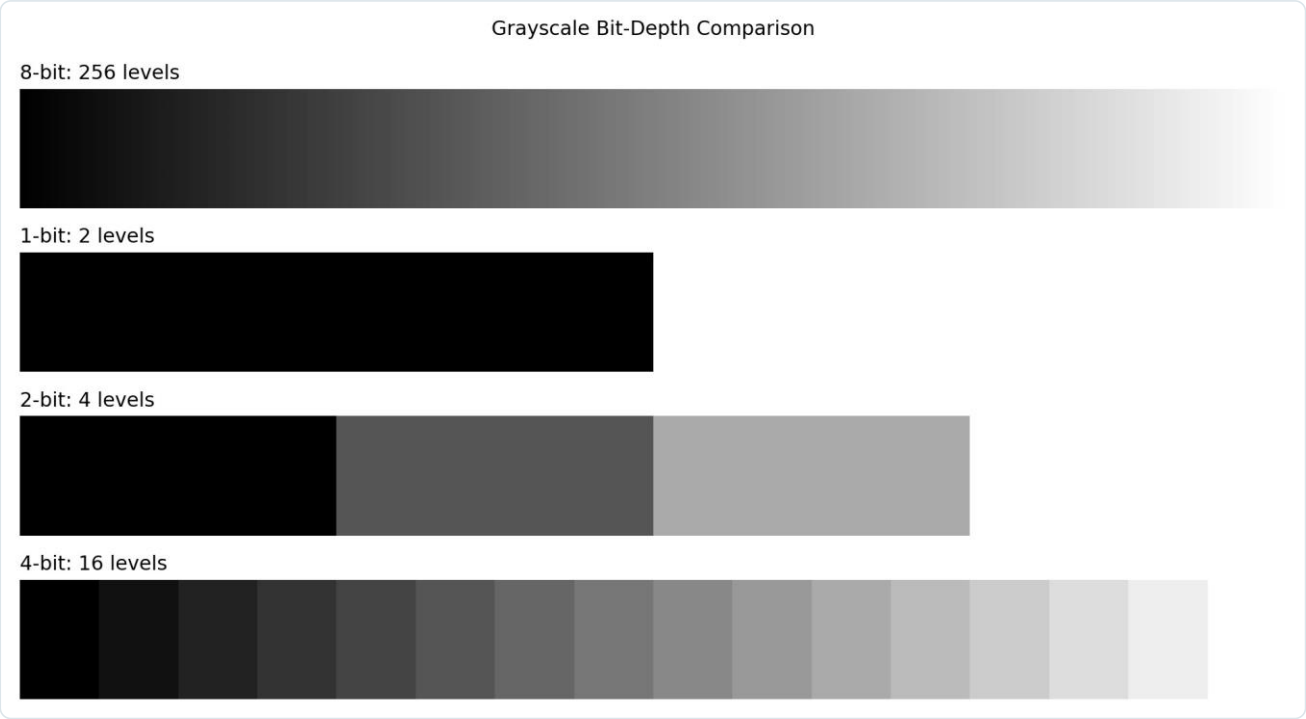


Figure. Horizontal grayscale gradient at 8-bit (256 levels), 1-bit (2), 2-bit (4), and 4-bit (16).

8-bit original	256 unique values
1-bit / 2-bit / 4-bit	2 / 4 / 16 unique values

RELEVANT SOURCE CODE - STEP 7

```
gradient = np.tile(np.arange(256, dtype=np.uint8),
                   (60, 1))
def quantize_to_bits(arr, bits):
    levels = 2 ** bits
    indices = (arr.astype(np.uint16) * levels) // 256
    return ((indices * 255) // (levels - 1)).astype(np.uint8)
for bits in (1, 2, 4):
    quantized = quantize_to_bits(gradient, bits)
    print(bits, len(np.unique(quantized)))
```

Resolution versus bit depth: Resolution specifies the number of pixels, usually expressed as width x height. Bit depth specifies how many intensity levels can be represented per pixel (for grayscale, 2^b levels with b bits). Lowering bit depth produces visible banding even when the image resolution remains unchanged.

07 / FINAL EXECUTION

Verification and deliverables

```
    Uninstalling pip-26.1.2:
      Successfully uninstalled pip-26.1.2
Successfully installed pip-26.2.2

(.venv) C:\Users\Admin\Desktop\dip-lab-02>python -c "import sys, cv2, numpy, matplotlib; print(sys.executable); print('All imports succe
C:\Users\Admin\Desktop\dip-lab-02\.venv\Scripts\python.exe
All imports successful

(.venv) C:\Users\Admin\Desktop\dip-lab-02>python lab02.py leisure.jpg

=== Step 2: Image Loading ===
Input image: leisure.jpg
Image shape (height, width, channels): (1536, 1024, 3)
Image data type: uint8
```

Figure. Student terminal screenshot: imports successful and lab02.py started with Leisure.jpg.

```
=== Step 7: Bit Depth ===
8-bit unique intensity count: 256
1-bit unique intensity count: 2
2-bit unique intensity count: 4
4-bit unique intensity count: 16
Resolution is the number of pixels (width x height); bit depth is the number of bits used to represent each pixel inten
Saved figure: bit_depth_comparison.png

LAB 02 COMPLETED SUCCESSFULLY. Close the figure windows to return to the terminal.

(.venv) C:\Users\Admin\Desktop\dip-lab-02>
```

Figure. Student terminal screenshot: unique-value counts (256, 2, 4, 16) and LAB 02 COMPLETED SUCCESSFULLY.

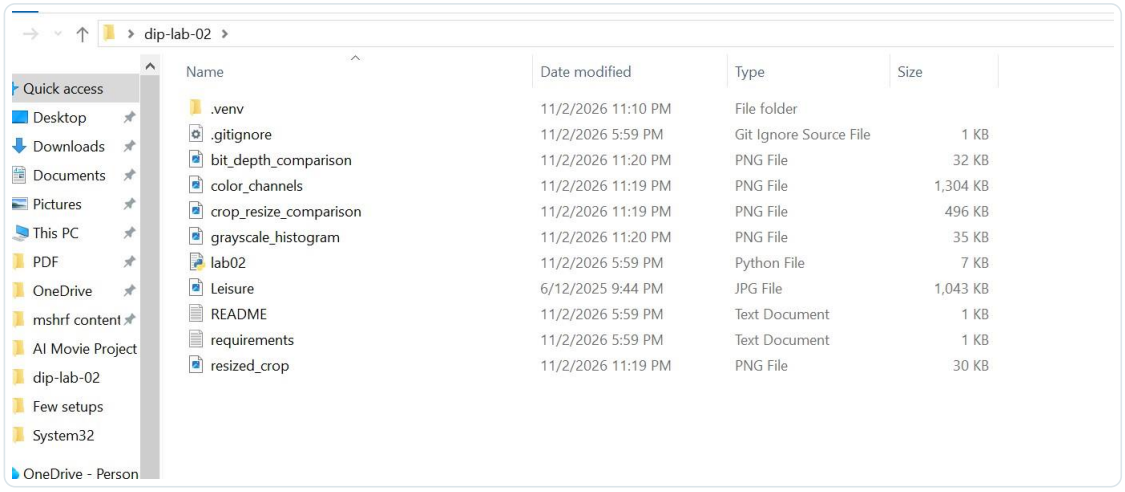


Figure. Student File Explorer screenshot: required project files present; .venv remains local and must not be submitted.

Submission files I attached: `lab02.py`, `Leisure.jpg`, `resized_crop.png`, `grayscale_histogram.png`, `.gitignore` and this PDF report. I kept `color_channels.png`, `crop_resize_comparison.png` and `bit_depth_comparison.png` as useful additional evidence.

Conclusion: Image loading, pixel inspection, BGR-to-RGB conversion, channel separation, cropping, resizing, histogram plotting and bit-depth comparison were completed successfully.